Monitor Cross-Review — Findings

Review of `contracts/utils/monitor/` against the sf-monitor design, following the September 2026 unbacked-future-value incident.

1 OCTOBER 2026 SF-MONITOR · SECURED-FINANCE/CONTRACTS

FOR CROSS-CHECK WITH THE CONTRACTS-REPO SESSION

1 Correction to an earlier assertion

> I previously confirmed that per-maturity future-value conservation holds **exactly at all times, with no tolerance band**. That is wrong. `utils/monitor/detectors.ts` is right to carry a pending term.

`contracts/protocol/libraries/logics/LendingMarketUserLogic.sol:219–221`

```
Storage.slot().pendingOrderAmounts[_ccy][_maturity] +=
    filledAmount - partiallyFilledOrder.amount;
```

Only the **partially** filled maker settles synchronously via `updateFundsForMaker`. **Fully filled makers settle lazily**, so lend and borrow supplies legitimately differ between fill and cleanup. I traced `_updateFunds` and the partially-filled path and over-generalised from it. The deferred path I missed is the same one the exploit abused.

Had we shipped a `!= 0` threshold it would have fired on ordinary traffic and been disarmed — while wired to auto-pause.

2 Verified mechanics worth building on

All checked directly in source:

- `FutureValueVault.sol:336–364` — `totalLendingSupplies` $= \Sigma \max(\text{balance}, 0)$, `totalBorrowingSupplies` $= \Sigma \max(-\text{balance}, 0)$. So `lend == borrow` $\iff \Sigma$ signed balances $== 0$.
- **The fee is taker-only.** `updateFundsForMaker` passes `_feeInFV = 0` (`FundManagementLogic.sol:279`). The reserve fund's future-value leg is **inside** the balancing set — excluding it as "protocol-internal" produces a false gap of exactly `feeInFV` on every taker fill.
- **Log-based checks must filter to `from == address(0)`.** `increase` / `decrease` emit matched mint/burn legs from the zero address, but the peer-to-peer path (`FutureValueVault.sol:242`) emits a *single* nonzero-value `Transfer`. Summing all `Transfer` values breaks on every position transfer.
- **`OPERATOR_ROLE` gates exactly six functions**, enumerated across the whole protocol — `TokenVault.pause/unpause`, `ReserveFund.pause/unpause`, `LendingMarketController.pauseLendingMarket/unpauseLendingMarket`. Nothing else. Note it includes *unpause*, so a compromised operator key can also lift a deliberate pause.

3 New finding — a second conservation law, unmonitored

`GenesisValueVault.sol:677–697` maintains `totalLendingSupplies[ccy]` / `totalBorrowingSupplies[ccy]` using **identical netting logic** to FutureValueVault, but per-currency rather than per-maturity.

That is an independent invariant covering the layer future-value conservation does not reach: auto-roll, compound-factor application, genesis-value transfers. It is the layer where one bad number reprices every

position in a currency simultaneously.

> **Caveat, given §1.** Before trusting a `!= 0` threshold here, check whether genesis value has its own lazy/deferred settlement path. The pattern appears in more than one place in this codebase — and that is exactly the question I failed to ask about future value.

---

4 Review of `utils/monitor/`

The detector logic is sound and correctly handles the pending term. Issues found:

1. **`trackMakers` collects takers, not makers** (`runner.ts:153–163`) — it reads `args.user` from `OrderExecuted` / `PreOrderExecuted`, i.e. whoever executed. Detector 3 needs holders of *resting* orders. `OrderExecuted` carries `placedOrderId` / `placedAmount`; filtering on `placedOrderId != 0` identifies makers precisely. As written, Detector 3 misses the exact profile it targets — place a resting order, return later to trigger it — whenever placement predates process start or falls outside the 200-entry window.
2. **No cursor persistence** (`runner.ts:52`) — `cursor` starts at head, so restarts silently skip every block during downtime. `recentMakers` is in-memory too.
3. **No reorg handling** — `scanBlock` walks forward with no confirmation depth. Armed, a reorg can pause on a block that no longer exists.
4. **Detector 2 is CRITICAL and auto-pauses, but it is an economic heuristic** — `alerts.ts` states heuristics should be HIGH and page a human. A floor of 5,000 auto-pauses any settlement below half par. Either demote it, or lower the floor substantially.
5. **Detector 1's tolerance mixes units** — `pendingOrderAmount` is present value, the supply gap is future value; `maxPendingFvMultiplier: 2` bridges them by assuming unit price ≥ 5,000. The band loosens as maturity lengthens, and an attacker staying under `2 × pendingAmount` evades it. Exactness is recoverable by tracking pending *future value* from fill events rather than inferring it from a present-value counter.
6. **No latch on the responder** — a persisting condition re-attempts the pause every sweep; the revert lands in `error` and logs FAILED repeatedly.

---

5 Implications now the bug is fixed

Invariants generalise; signatures do not. Detectors 2 and 3 are shaped around this attack's mechanism — the next bug may not produce an absurd price. They are cheap and worth keeping as backstops, but Detector 1's style should be the foundation.

Candidate invariant suite

INVARIANT	STATUS & COVERAGE
Future-value conservation, per maturity	Implemented, correctly, with pending term
Genesis-value conservation, per currency	Unmonitored — see §3; covers the post-auto-roll layer
Custody solvency	`balanceOf(TokenVault)` vs `getTotalDepositAmount(ccy)`
Order-book structural integrity	Every order in a user's active list reachable via `isActiveOrderId` — fires *before* value moves
Compound-factor sanity per auto-roll	Unmonitored; bounded movement between rolls

Follow-on consequences

- **State-based checks are more upgrade-robust than log-based ones.** A new code path that mints future value still breaks conservation; it may not emit the event you were watching.
- **Silence should be an alert.** A monitor matching nothing for an unexpected window may be calm — or orphaned by an upgrade or a renamed event.
- **Not covered by the current design at all: version and bytecode drift.** "Is the deployed code what we wrote?" The September fix was merged 2026-06-17 (`f3b7a169`) and was still not live on 2026-09-05. The cheap half needs no RPC — compare the published `@secured-finance/contracts` version against contracts-repo main.
- **Test strategy:** synthetic violation of each invariant on a fork (assert it fires), plus a **clean soak against real forked traffic asserting zero alerts**. The soak matters more — it is what would have caught the §1 error before shipping rather than after.

6 Architecture

The detector logic is the valuable artifact and should be kept. The runner is a prototype rather than a service — no cursor persistence, no reorg handling, console output only, one chain per invocation, and an RPC hiccup kills the loop. sf-monitor already has that spine: cursor persistence, multi-network, Slack routing, container and GKE deployment.

Suggested split:

- **Detection** ports to sf-monitor.
- **The responder** becomes its own minimal keyholder process — it should not live in either container.
- **The hardhat task stays** as the fork-replay harness. That is the one thing sf-monitor cannot easily do, and it is where both the synthetic-violation and soak tests belong.

7 Open questions for the contracts-repo session

1. Does genesis value have a lazy/deferred settlement path analogous to `pendingOrderAmounts`?
2. Is `maxPendingFvMultiplier: 2` the intended long-term design, or a placeholder until pending future value can be tracked exactly from fill events?

3. Was Detector 2 being CRITICAL and auto-pausing deliberate, given `alerts.ts`'s own stated policy for heuristics?
4. Was version and bytecode drift consciously out of scope, or simply not considered?
5. Intended runtime home — hardhat task, or ported to sf-monitor's existing deployment?

Source references are to `secured-finance/contracts` at the working tree reviewed on 1 October 2026. All claims in §2 were verified directly against source; §5 is judgement, not verification.